# Lab 6: TCP Connection Management, Flow Control, and Congestion Control

**Lab Name:** Lab 06: TCP Connection, Flow, and Congestion Control
**Software/Hardware Used:** Cisco Modeling Labs (CML) - Personal / Free Edition

---

## AI Policy & Acceptable Use Guidelines

 AI Policy & Acceptable Use Guidelines - Yellow Icon

Artificial Intelligence tools can be powerful learning aids, but they must be used ethically and responsibly to ensure you develop core networking skills.

- **Lab Assignments:** Do not use AI to perform lab steps or write your lab submissions. However, if you get stuck on a concept or technical obstacle, you may use AI as a virtual tutor to ask clarifying questions and help you troubleshoot.

---

## 1. Background

The Transmission Control Protocol (TCP) is the workhorse transport protocol of the Internet, providing reliable, in-order, and congestion-controlled byte stream delivery between end systems. To achieve high performance while maintaining network stability, TCP coordinates four interconnected mechanisms:

1. **Connection Management**: A three-way handshake ( `SYN` , `SYN-ACK` , `ACK` ) establishes sequence numbers and socket state before data transmission, followed by a four-way teardown ( `FIN` , `ACK` ) to gracefully release resources.
2. **Reliable Data Transfer (RDT)**: Sequence numbers track each byte of payload data, Cumulative and Selective Acknowledgments (SACK) confirm receipt, and fast retransmissions / timeouts recover from lost segments.
3. **Flow Control (End-System Bottleneck)**: A sliding-window protocol where the receiver continuously advertises its available buffer capacity (**Receive Window / `rwnd`** ). The sender guarantees that unacknowledged bytes in flight never exceed `rwnd` , preventing a fast sender from overflowing a slow receiver's buffer.
4. **Congestion Control (Network Bottleneck)**: An Additive-Increase Multiplicative-Decrease (**AIMD**) algorithm dynamically adjusts the sender's **Congestion Window ( `cwnd` )**. The sender starts in **Slow Start** (exponential growth), transitions to **Congestion Avoidance** (linear probing), and slashes its rate upon detecting packet loss.

A common point of confusion is distinguishing **Flow Control** (receiver-limited) from **Congestion Control** (network-limited). In this lab, you will generate sustained TCP traffic across a constrained CML network, isolate both flow and congestion bottlenecks, and analyze TCP mechanics using Wireshark's advanced `tcptrace` graphing engine and `iperf` .

### Raw vs. Relative Sequence and Acknowledgment Numbers

When inspecting TCP packets in Wireshark, you will encounter two representations of sequence and acknowledgment numbers:

- **Raw Sequence / Acknowledgment Numbers (On-the-Wire Headers):** These are the actual 32-bit unsigned integer values transmitted directly within the TCP header over the physical network link. During connection establishment (the 3-way handshake), each endpoint randomly generates an **Initial Sequence Number (ISN)**—a 32-bit value between $0$ and $2^{32}-1$ (e.g., `2839401283` ). Randomizing the ISN

prevents TCP sequence prediction attacks (session hijacking) and prevents delayed segments from previous, closed connections from being misinterpreted as valid data in a new session. Because raw numbers are randomly initialized 32-bit integers, they appear as large 10-digit numbers.

- **Relative Sequence / Acknowledgment Numbers (Wireshark Convenience):** Tracking arithmetic and byte offsets across large 10-digit random numbers can be cumbersome during packet analysis. To simplify analysis, Wireshark enables **Relative Sequence Numbers** by default. Wireshark sets the client's initial `SYN` packet sequence number to **$0$** ($ISN_{relative} = 0$). Subsequent sequence and acknowledgment numbers then represent the exact cumulative byte offset from the start of the stream (e.g., Sequence Number `1`, `1441`, `2881`, etc.).
- **Viewing in Wireshark:** In Wireshark's packet details pane, expand the **Transmission Control Protocol** tree:
  - `Sequence Number: 1 (relative sequence number)` displays the Wireshark relative index.
  - `Sequence Number (raw): 2839401284` displays the actual 32-bit wire value.
  - You can toggle this setting globally in Wireshark under **Edit -> Preferences -> Protocols -> TCP -> "Relative sequence numbers"**.

---

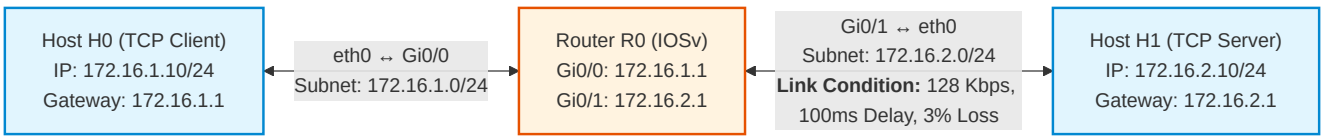
## 2. Objective / Purpose

The objectives of this lab are to:

- Trace and analyze the TCP 3-Way Handshake and Connection Teardown.
- Observe **TCP Congestion Control in action**, triggering the classic AIMD "sawtooth" pattern through packet loss and link capacity limits.
- Experimentally isolate **Flow Control** (Receiver Bottleneck) from **Congestion Control** (Network Bottleneck) by manipulating Linux receive buffer sizes ( `sysctl` ).
- Use Wireshark's **`tcptrace` graph** to simultaneously visualize in-flight sequence numbers, acknowledgments, and the Advertised Receive Window.
- Benchmark real-time TCP throughput and bandwidth stability using **`iperf`**.

---

## 3. Topology Diagram




---

## 4. IP Addressing Table

| Device | CML Node Name | Interface | IP Address | Subnet Mask | Default Gateway | Subnet |
|---|---|---|---|---|---|---|
| **Client** | `<initials>-H0` | `eth0` | `172.16.1.10` | `255.255.255.0` (`/24`) | `172.16.1.1` | `172.16.1.0/24` (Subnet 1) |
| **Server** | `<initials>-H1` | `eth0` | `172.16.2.10` | `255.255.255.0` (`/24`) | `172.16.2.1` | `172.16.2.0/24` (Subnet 2) |
| **Router** | `<initials>-R0` | `Gi0/0` | `172.16.1.1` | `255.255.255.0` (`/24`) | N/A | `172.16.1.0/24` (Subnet 1) |

| | | | | | | |
|---|---|---|---|---|---|---|
| **Router** | `<initials>-R0` | `Gi0/1` | `172.16.2.1` | `255.255.255.0` (`/24`) | N/A | `172.16.2.0/24` (Subnet 2) |

---

## 5. Network Configuration & Environment Setup

> *[!IMPORTANT] **New Subnets for Lab 6 — Reconfigure Your Network!** You must configure both hosts and the router with the `172.16.1.0/24` and `172.16.2.0/24` subnets. Refer to previous labs for the exact Linux host commands ( `ip addr add` , `ip link set eth0 up` , `ip route add default via ...` ) and Cisco router interface configuration commands.*
>
> *Verify connectivity before proceeding: Ensure H0 can ping its gateway ( `ping -c 3 172.16.1.1` ) and ping across the router to H1 ( `ping -c 3 172.16.2.10` ).*

### Configuring CML Link Impairments (The Bottleneck Link)

To force TCP to clearly exhibit its congestion control, slow-start, and retransmission dynamics, configure link conditioning on the link between **Router R0 and Host H1**:

1. In the CML canvas, click on the **link connecting R0 and H1**.
2. Navigate to the **Link Condition** tab in the lower pane and configure a heavily constrained link:
   - **Bandwidth**: `128` Kbps
   - **Latency**: `100` ms (200 ms Round-Trip Time)
   - **Packet Loss**: `3.0` % (triggers clear AIMD multiplicative decrease and fast recovery events)
3. Click **Set Condition**.

---

## 6. Lab Execution and Experimental Procedures

> *[!IMPORTANT] **REMINDER: All screenshots, console outputs, and packet captures (.pcap files) must be taken from the SAME active lab session. Mixing screenshots or trace files across different runs or sessions is not permitted.***

### Preparation: Generate Numbered Test Payload with Custom Trailer (Host H0)

To make it easy to visually track data transfer in real time across the console, generate a 150KB payload file on Host H0 consisting of 3,000 sequentially numbered lines, ending with a custom completion trailer containing your initials:

```
# On Host H0: Generate 3,000 numbered payload lines (~150KB total)
seq -w 1 3000 | sed 's/.*/Line &: CS457 TCP Packet Stream Payload Segment/' > alice.txt

# Append custom completion trailer marker
echo "=== CS457 LAB6 FILE TRANSFER COMPLETE: <INITIALS> ===" >> alice.txt
```

*(For example, student John Doe would type `echo "=== CS457 LAB6 FILE TRANSFER COMPLETE: JD ===" >> alice.txt` ). You can verify with `ls -lh alice.txt` and `wc -l alice.txt` (3,001 lines).*

---

### Experiment 1: Network Bottleneck & Congestion Control (Run 1)

In this run, the receiver buffer is unconstrained, meaning the **heavily constrained network link** (128 Kbps bandwidth, 100ms latency, and 3% packet loss) is the sole bottleneck.

1. **Start Packet Capture**: In CML, click on the link between **Host H0 and Router R0**, select the **Packet Capture** tab, set the filter to `tcp port 8080`, and click **Start**.
2. **Start Receiver on Host H1 (Streaming Live to Console & Saving to File)**:

```
nc -v -l -p 8080 | tee received_run1.txt
```

3. **Transmit File from Host H0**:

```
nc -v 172.16.2.10 8080 < alice.txt
```

4. **Monitor & Conclude**:
   - **On H1's Console**: You will see the incremental line numbers stream down the screen over the course of **~10–12 seconds** ( `Line 0001: ...` up to `Line 3000: ...` ), concluding with the completion trailer:

```
Line 2998: CS457 TCP Packet Stream Payload Segment
Line 2999: CS457 TCP Packet Stream Payload Segment
Line 3000: CS457 TCP Packet Stream Payload Segment
=== CS457 LAB6 FILE TRANSFER COMPLETE: <INITIALS> ===
```

   - Once transmission finishes, press `Ctrl+C` on H0 (and H1 if needed) to close netcat.
   - Stop the packet capture in CML, download it, and rename it **`tcp_run1_congestion.pcap`**.
   - Verify on H1: `ls -lh received_run1.txt` (~150KB) and `wc -l received_run1.txt` (3,001 lines).

---

### Experiment 2: Throughput Benchmarking with `iperf`

With the 128 Kbps / 3% packet loss link condition still active, use `iperf` to benchmark sustained network throughput and observe periodic bandwidth reports.

1. **Start `iperf` Server on Host H1**:

```
iperf -s
```

2. **Run 15-Second Benchmark from Host H0**:

```
iperf -c 172.16.2.10 -t 15 -i 1
```

   *Note: The `-t 15` flag sets the test duration to 15 seconds, and `-i 1` displays bandwidth reports at 1-second intervals.*
3. **Record Results**:

> ***IMPORTANT - Deliverable Screenshot**: Take a screenshot of the `iperf` terminal output on Host H0 showing the 1-second interval bandwidth reports and final transfer summary.*

---

### Experiment 3: Receiver Bottleneck & Flow Control (Run 2)

To isolate **Flow Control** from network congestion, we will remove the network link constraints (restoring full link capacity) and artificially strangle the **TCP Receive Buffer** on Host H1 using Linux kernel parameters ( `sysctl` ). This proves that the *receiver's advertised window ( `rwnd` )*, rather than network packet drops, is throttling the sender.

1. **Remove Link Conditioning in CML (Restore Clean Link)**:

- In the CML canvas, click on the link between **Router R0 and Host H1**.
- Navigate to the **Link Condition** tab and click **Reset** (or clear all bandwidth, latency, and packet loss impairments).

2. **Constrain Receive Buffer on Host H1**: On H1's console, set the TCP receive buffer limits (min, default, max) to very small values:

```
sysctl -w net.ipv4.tcp_rmem="4096 8192 16384"
```

3. **Start Packet Capture in CML**: On the link between **Host H0 and Router R0**, start a new packet capture ( `tcp port 8080` ).
4. **Start Receiver on Host H1**:

```
nc -v -l -p 8080 | tee received_run2.txt
```

5. **Transmit File from Host H0**:

```
nc -v 172.16.2.10 8080 < alice.txt
```

6. **Observe, Stop & Restore**:
   - Notice how the transfer rate on H1's console is clamped by the receiver's tiny buffer window, even though the network link is now unconstrained.
   - Once transmission completes, press `Ctrl+C` to close netcat.
   - Stop the CML packet capture, download it, and rename it **`tcp_run2_flowcontrol.pcap`** .
   - **Restore Default Buffers on H1**:

```
sysctl -w net.ipv4.tcp_rmem="4096 131072 6291456"
```

---

# 7. Wireshark Analysis and Questions

### Part A: TCP Basics and Handshake (from `tcp_run1_congestion.pcap` )

1. **Client & Server Sockets**: What IP address and TCP port number are used by `<initials>-H0` ? What IP address and port are used by `<initials>-H1` ?
2. **SYN Segment Analysis**: Locate the initial TCP SYN segment:
   - What is the raw Sequence Number?
   - What TCP flags are set?
   - What Maximum Segment Size (MSS) and Window Scale factor are requested in the TCP Options?
3. **SYN-ACK Segment Analysis**: Locate the SYN-ACK response from H1:
   - What are its Sequence Number and Acknowledgment Number? Specify both the **raw** values (actual wire headers) and the **relative** values (Wireshark normalized display).
   - Explain the mathematical relationship between the SYN-ACK's Acknowledgment number and the client's initial Sequence Number ($\text{ACK} = \text{Client ISN} + 1$), demonstrating the arithmetic calculation using both raw and relative numbers.
4. **Connection Teardown**: Locate the connection termination packets (FIN/ACK sequence). Outline the packet numbers and sequence of flags exchanged to close the socket.

---

### Part B: Reliable Data Transfer & Retransmissions (from `tcp_run1_congestion.pcap` )

5. **Sequence & Acknowledgment Tracking**: Inspect the first two data-carrying segments sent by H0:
    - What is the Sequence Number (specify both **raw** and **relative**) and the payload byte count (TCP segment length) of the first data segment?
    - What is the Acknowledgment number (specify both **raw** and **relative**) sent back by H1 in response to this first segment? Show the calculation using the formula: $\text{ACK} = \text{Seq} + \text{Payload Bytes}$ (demonstrating how this relationship holds for both raw and relative sequence tracking).
6. **Packet Loss & Fast Retransmit**: Filter Wireshark for `tcp.analysis.flags`:
    - Identify an instance of a **TCP Duplicate ACK** and a subsequent **TCP Fast Retransmission**.
    - What packet numbers correspond to the duplicate ACKs and the retransmitted segment?
    - How does Selective Acknowledgment (SACK) help the sender identify which exact segment was lost?

---

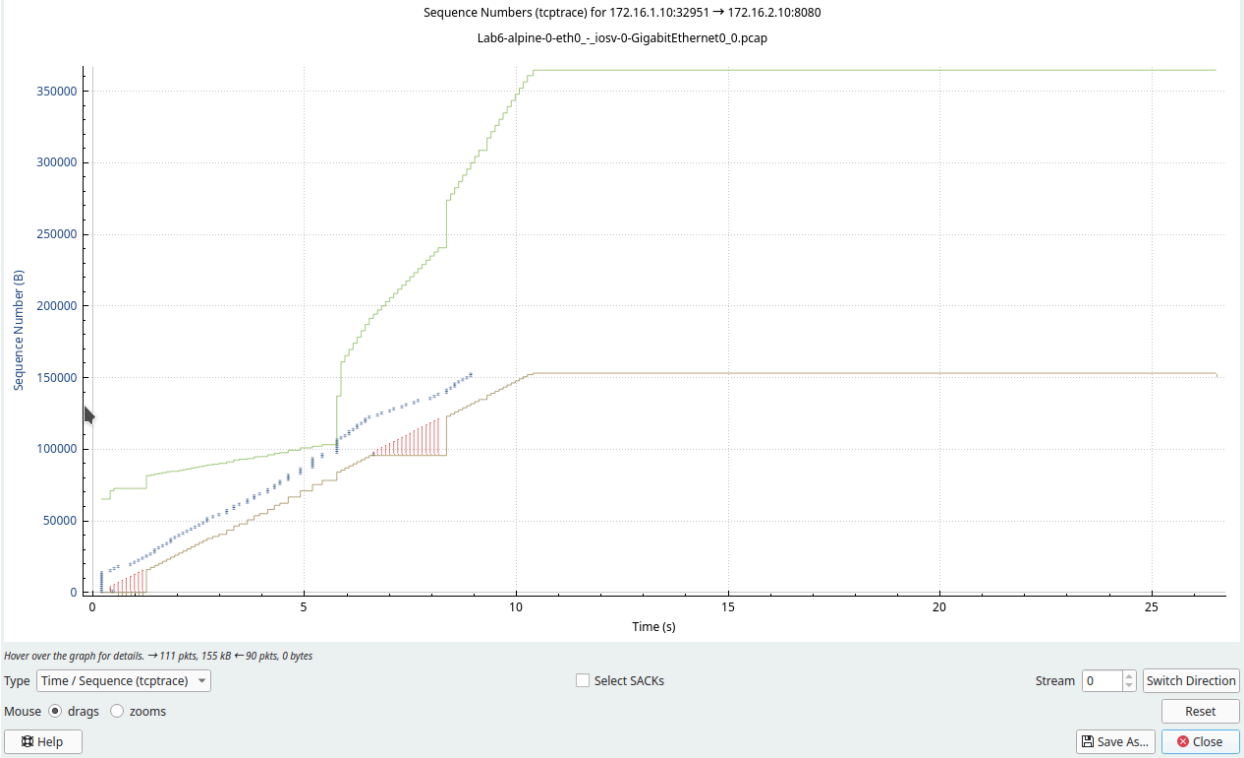
### Part C: Flow Control vs. Congestion Control (`tcptrace` Graphing)

Wireshark's **`tcptrace`** graph (`Statistics -> TCP Stream Graph -> Time Sequence Graph (tcptrace)`) simultaneously plots:

- **Lower line / data points**: Sequence numbers sent (data in flight).
- **Middle line**: Cumulative Acknowledgments received.
- **Upper line**: Advertised Receive Window ($\text{ACK} + \text{rwnd}$).

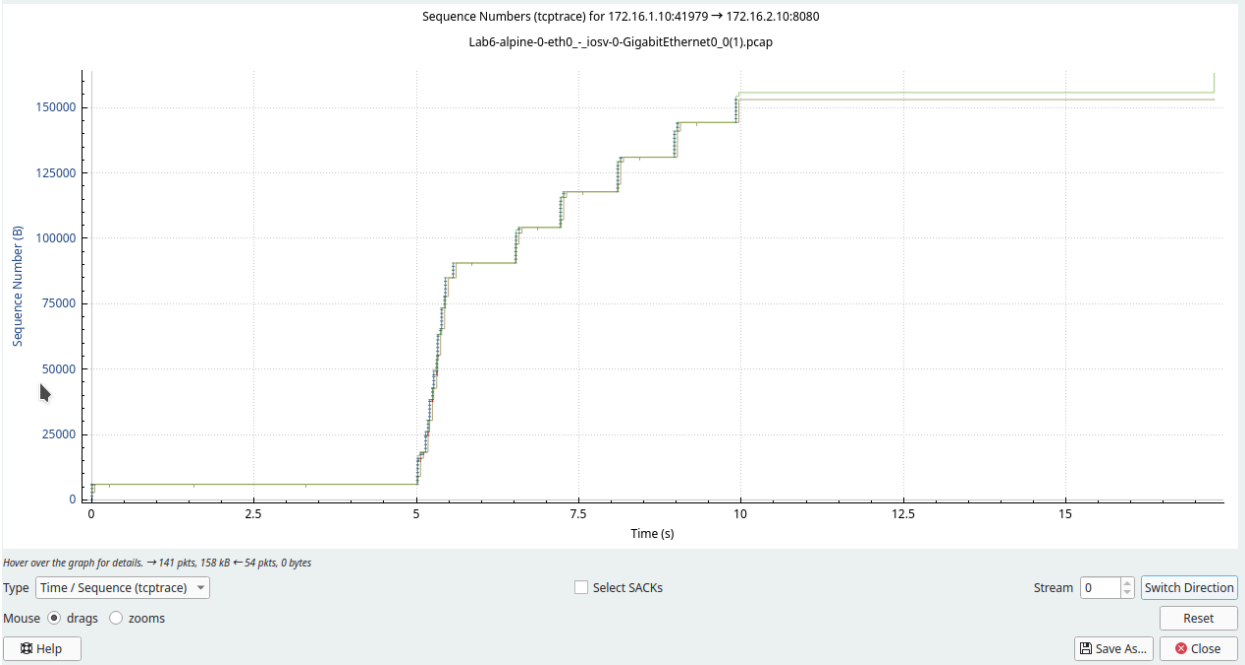
> *[!IMPORTANT] **Switching Stream Direction in the tcptrace Graph:** When you open the `tcptrace` graph, Wireshark may default to displaying the return traffic (from Server H1 back to Client H0), which contains only flat ACK packets without data sequence growth.*
>
> - ***Action Required**: If your graph appears blank, completely flat, or shows no sequence progression, click the "**Switch Direction**" button at the bottom of the graph window (or select a data packet originating from `172.16.1.10` in the main packet list before opening the graph).*
> - *Verify that the graph title bar at the top indicates the outbound flow from the client to the server: `172.16.1.10:<port> → 172.16.2.10:8080`.*

7. **Run 1 `tcptrace` Analysis (Network Bottleneck / Congestion Control)**: Open `tcp_run1_congestion.pcap`, select a data packet sent from H0, and open `Statistics -> TCP Stream Graph -> Time Sequence Graph (tcptrace)`. Click **Switch Direction** if needed.




- **Annotated Deliverable**: Capture a screenshot of this graph.
- Describe the shape of the data curve: identify where the connection is in **Slow Start** (exponential curve) versus **Congestion Avoidance** (linear probing).
- Point out where packet loss occurs: what happens to the slope of the sequence numbers when loss is detected? Explain the AIMD (Additive Increase, Multiplicative Decrease) behavior observed.
- Is the data line touching the upper Receive Window line, or is it well below it? What does this tell you about whether the network or the receiver is the bottleneck?

8. **Run 2 `tcptrace` Analysis (Receiver Bottleneck / Flow Control)**: Open `tcp_run2_flowcontrol.pcap`, select a data packet, and open `Statistics -> TCP Stream Graph -> Time Sequence Graph (tcptrace)`. Click **Switch Direction** if needed.




- **Annotated Deliverable**: Capture a screenshot of this graph.
- How does the Advertised Receive Window line in Run 2 differ from Run 1?
- Explain how the constrained receive buffer ( `sysctl tcp_rmem` ) forced the sender's transmission rate to stall, even when network bandwidth was available.

---

### Part D: Throughput Benchmarking Analysis (from `iperf` )

9. **Throughput vs. Capacity**: Based on your `iperf` output, what was the average transfer bandwidth achieved over the 15-second test? How close was this to the 128 Kbps link limit?
10. **Link Constraint Impact**: How did the 128 Kbps bandwidth and 3% packet loss constraints affect the throughput reported by `iperf` compared to an unconstrained link?

---

## 8. Troubleshooting

**Issue Encountered:** [Describe what went wrong, if anything]
**Discovery Method:** [How did you identify the issue?]
**Resolution:** [How did you resolve it?]

---

## 9. What to Hand In

Submit the following deliverables:

### 1. Configuration File

- The Lab YAML export configuration file with your name, date, and lab title added at the top in comments.

### 2. Wireshark Packet Capture Files

- Both packet capture files from the experiment runs:
  - `tcp_run1_congestion.pcap` (or `.pcapng` ) (Run 1: Unconstrained receiver / Congestion Control)

- `tcp_run2_flowcontrol.pcap` (or `.pcapng`) (Run 2: Constrained receiver / Flow Control)

## 3. Comprehensive Lab Report (Single PDF Document)

Your submitted PDF lab report must contain:

**A. Written Analysis Answers**

- Complete, accurate written answers to all 10 analysis questions in Section 7 (Parts A through D).
- Complete Troubleshooting (Section 8) sections.

**B. Required Screenshots & Explicit Annotation Guidelines:**

> ***IMPORTANT - Annotation Standards**: Every screenshot submitted for grading must include clear visual annotations (e.g., callout boxes, arrows, circles, and text labels). Submitting unannotated screenshots will result in a point deduction.*

1. **CML Topology & Link Condition Screenshot**:
   - Screenshot showing active CML workspace with nodes `<initials>-H0`, `<initials>-R0`, `<initials>-H1`, and the link condition pane showing **128 Kbps, 100ms latency, and 3% packet loss**.
2. **Annotated TCP 3-Way Handshake Screenshot (Part A Q2 & Q3)**:
   - Wireshark packet list pane showing the SYN, SYN-ACK, and ACK packets.
   - **Annotations Required**: Label packets, annotate ISN and ACK $+1$ relation, and highlight TCP flags and MSS/Window Scale options.
3. **Annotated `tcptrace` Graph — Run 1: Congestion Control (Part C Q7)**:
   - Wireshark `tcptrace` window from `tcp_run1_congestion.pcap` *(ensure graph direction is set from Client H0 `172.16.1.10` to Server H1 `172.16.2.10` via "Switch Direction")*.
   - **Annotations Required**:
     - Label the **Slow Start** phase and **Congestion Avoidance** linear slope.
     - Highlight packet loss / AIMD multiplicative decrease steps.
     - Note the gap between data in flight and the Advertised Receive Window line.
4. **Annotated `tcptrace` Graph — Run 2: Flow Control (Part C Q8)**:
   - Wireshark `tcptrace` window from `tcp_run2_flowcontrol.pcap` *(ensure graph direction is set from Client H0 `172.16.1.10` to Server H1 `172.16.2.10` via "Switch Direction")*.
   - **Annotations Required**:
     - Highlight the low Advertised Receive Window line clamping the data sequence rate.
     - Contrast this directly with Run 1 to illustrate receiver-driven flow control.
5. **`iperf` Throughput Benchmark Screenshot (Part D Q9 & Q10)**:
   - Console screenshot on Host H0 displaying the full 15-second `iperf` output showing the 1-second interval bandwidth reports and final summary table.